

PRACTICAL WEEK 12

1. Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical12.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>

#define ITEMS 100000

int *buffer;
int buffer_size;

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 0; i < ITEMS; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    long long sum = 0;

    for (int i = 0; i < ITEMS; i++)
    {
```

```

        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        sum += buffer[out];
        out = (out + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);
    }

    printf("Consumer sum: %lld\n", sum);

    return NULL;
}

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Usage: %s <buffer_size>\n", argv[0]);
        return 1;
    }

    buffer_size = atoi(argv[1]);

    if (buffer_size <= 0)
    {
        printf("Invalid buffer size\n");
        return 1;
    }

    buffer = malloc(buffer_size * sizeof(int));

    if (buffer == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    pthread_t producer_thread;
    pthread_t consumer_thread;

```

```

sem_init(&empty, 0, buffer_size);
sem_init(&full, 0, 0);
pthread_mutex_init(&mutex, NULL);

struct timespec start, end;

clock_gettime(CLOCK_MONOTONIC, &start);

pthread_create(&producer_thread, NULL, producer, NULL);
pthread_create(&consumer_thread, NULL, consumer, NULL);

pthread_join(producer_thread, NULL);
pthread_join(consumer_thread, NULL);

clock_gettime(CLOCK_MONOTONIC, &end);

double elapsed = (end.tv_sec - start.tv_sec) +
                 (end.tv_nsec - start.tv_nsec) / 1000000000.0;

double throughput = ITEMS / elapsed;

printf("Buffer size: %d\n", buffer_size);
printf("Items processed: %d\n", ITEMS);
printf("Execution time: %.6f seconds\n", elapsed);
printf("Throughput: %.2f items/second\n", throughput);

sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

free(buffer);

return 0;
}

```

Run the code:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical12.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical12.1.c -o practical12.1 -pthread
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical12.1 10

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical12.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical12.1.c -o practical12.1 -pthread
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical12.1 10
Consumer sum: 4999950000
Buffer size: 10
Items processed: 100000
Execution time: 0.252778 seconds
Throughput: 395603.29 items/second
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

2. Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical12.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg)
{
    printf("Thread 1: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Resource 1 locked\n");
    sleep(1);

    printf("Thread 1: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Resource 2 locked\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Resource 2 locked\n");
    sleep(1);

    printf("Thread 2: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Resource 1 locked\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}
```

```

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Program completed\n");

    return 0;
}

```

Open second terminal and create a prevention file:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical12.2.prevention.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$

```

Write the code:

```

GNU nano 8.7.1 practical12.2.prevention.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg)
{
    printf("Thread 1: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Resource 1 locked\n");
    sleep(1);

    printf("Thread 1: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Resource 2 locked\n");

    printf("Thread 1: Using both resources\n");
    sleep(1);

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 1: Resources released\n");

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Resource 1 locked\n");
    sleep(1);
}

```

```

    printf("Thread 2: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Resource 2 locked\n");

    printf("Thread 2: Using both resources\n");
    sleep(1);

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 2: Resources released\n");

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("No deadlock occurred\n");

    return 0;
}

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical12.2.prevention.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical12.2.prevention.c -o pr
actical12.2.prevention -pthread
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical12.2.prevention
Thread 1: Locking Resource 1
Thread 1: Resource 1 locked
Thread 2: Locking Resource 1
Thread 1: Locking Resource 2
Thread 1: Resource 2 locked
Thread 1: Using both resources
Thread 1: Resources released
Thread 2: Resource 1 locked
Thread 2: Locking Resource 2
Thread 2: Resource 2 locked
Thread 2: Using both resources
Thread 2: Resources released
No deadlock occurred

```